DAA PROJECT

Course Instructor: Miss Anaum

Team

1. Saad 24K0549
2. Yash 24K0737
3. Abdullah 24K0977

DAA PROJECT

SECTION 01 (Done by Abdullah 24K0977)

PART A

An array A[0...n-1] containing $n \geq 1$ comparable numbers. It will output the minimum and the maximum values in the array

(a) Pseudocode

```
ALGORITHM MinMax(A, low, high) {
    //Returns min and max of array A
    if low == high then
        return (A[low], A[low])
    mid = (left + (right - left) / 2)
    (minLeft, maxLeft) = MinMax(A, low, mid)
    (minRight, maxRight) = MinMax(A, mid+1, high)
    if minLeft < MinRight
        overallMin = minleft
else if
        overallMin = minRight
    if maxLeft > maxRight
        overallmax = maxLeft
    else
        overallMax = maxRight

    return (overallMin, overallMax)
}
```

↳ The initial call will be :

(min, max) = MinMax(A, 0, n-1)

↳ The algorithm uses no sorting. It examines every element through the recursive subproblem and compares only the two returned minima and maxima.

↳ We can prove correctness by induction on the subarray size

① Base Case: for a single element in the subarray, it is both the min and the max.

② Inductive step: The minima of the entire subarray must be < the two half minima, and its max should be > the two half maxima. The combine step makes exactly these decisions and and returns the correct result for the subarray

(b) Recurrance

↳ Let $C(n)$ be the number of key comparisions performed by this algo. For $n = 1$, no comparision is needed:

$$C(1) = 0$$

↳ For $n > 1$ the algo makes two recursive calls on subarrays of size $n/2$, and compares, so:

$$C(n) = 2C(n/2) + 2, \quad n > 1$$

↳ now we solve the recurrance by expansion:

$$C(n) = 2C(n/2) + 2$$
$$C(n) = 2[2C(n/4) + 2] + 2$$
$$C(n) = 4C(n/4) + 6$$
$$C(n) = 8C(n/8) + 14$$

after i levels:

$$C(n) = 2^i C(n/2^i) + 2(1 + 2 + \dots + 2^{i-1})$$

since $n = 2^k$

$$i = k = \log_2 n$$
$$C(n) = 2^k C(1) + 2(1 + 2 + \dots + 2^{k-1})$$

Using $C(1) = 0$ & the geometric series identity:

$$1 + 2 + \dots + 2^{k-1}$$
$$\rightarrow 2^k - 1$$

we obtain: $C(n) = 0 + 2(2^k - 1)$

$$= 2(2^k - 1)$$
$$= 2n - 2$$

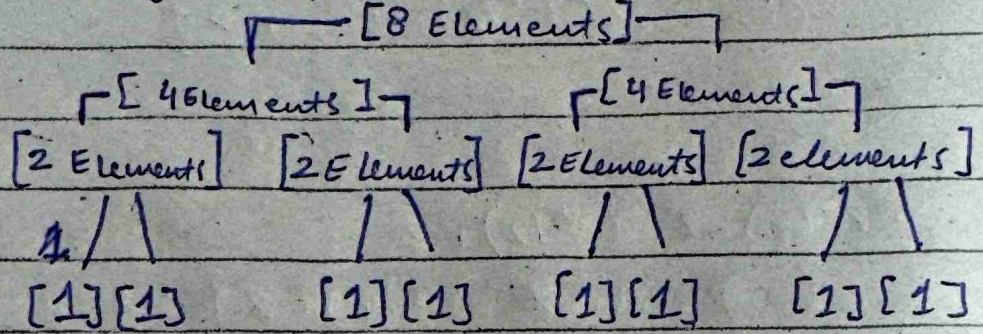
↳ Therefore for $n = 2^k$,

$$C(n) = 2n - 2$$

and the number of Key comparisions is $\Theta(n)$

↳ Recursion Tree

for $n = 8$, each node performs 2 Key comparisions:



@ level 0 → 1 internal node

@ level 1 → 2 " 4

@ level 2 → 34 " 4

Total Internal Nodes:

$1 + 2 + 4 = 7$

$= n - 1$

A full binary recursion tree produces $n-1$ internal nodes. With 2 comparisons per node, the total becomes $2(n-1)$

$= 2n - 2$

(c) Comparision with Brute Force

↳ BF initialises both min & max to the 1st element and then scans the remaining n-1 elements. This results in exactly

$$(n-1) + (n-1) = 2n-2$$

Key comparisions, with a time complexity of $\Omega(n)$.

Hence, the divide and conquer approach does not reduce the total number of key comparisions.

PART B (Done by Abdullah 24K0977)

$$a^n = \begin{cases} (a^{n/2})^2, & \text{if } n = \text{even} \\ a(a^{(n-1)/2})^2, & \text{if } n = \text{odd} \end{cases}$$

(a) Pseudocode

```
Algorithm Power(a, n) {
    // computes a^n for n >= 1 using
    // divide and conquer
    if n=1 then
        return a
    halfPower = Power(a, floor(n/2))
    square = halfPower * halfPower
    if n is even then
        return square
    else
        return a * square
}
```

(b) Recurrance

↳ Let $M(n)$ = the number of multiplications performed by Power(a, n). The base case would return a w/o multiplying:

$$M(1) = 0$$

↳ Every recursive call computes half power once using exponent $\lfloor n/2 \rfloor$. Hence

① if n = even → one multiplication squares half power

② if n = odd AND $n > 1$ → two multiplications are performed; one to square the half power and the other to multiply it by a.

↳ Hence the exact recurrance is:

$$M(n) = \begin{cases} 0 & , \ n = 1 \\ M(n/2) + 1 & , \ n > 1 \ \& \ \text{even} \\ M((n-1)/2) + 2 & , \ n > 1 \ \& \ \text{odd} \end{cases}$$

↳ Solving recurrance for $n = 2^k$

$M(n) = M(n/2) + 1$, $M(1) = 0$

$M(n) = M(n/4) + 2$

$M(n) = M(n/8) + 3$

after i levels

$M(n) = M(n/2^i) + i$

now: $i = k = \log_2 n \rightarrow n/2^k = 1$, so:

$M(n) = M(1) + k = 0 + \log_2 n$

Hence for $n = 2^k$

$M(n) = \log_2 n$

with a time complexity of $\Theta(\log n)$

© Comparision with Brute Force

↳ BF algo initializes a result to "a" and multiplies by "a" another $n-1$ times.

↳ For $n=1$, the loop does not execute. For any $n>1$, the multiplication count is $B(n) = n-1$, and TC is $\Theta(n)$.

↳ The divide and conquer exponentiation is asymptotically more efficient in the number of multiplications. It reduces the exponent by roughly half at each recursive call.

DAA Project Deliverables Date:

M. Saad Sohail
24k-0549

Pt 1

Q1 c) we will count the number of count inversions using merge sort algorithm.

~~(First, divide the array into two halves.)~~

```
FUNCTION CountInversions (Array A, int low, int high):
    if low >= high:
        return 0
    mid = low + high //2
    leftCount = CountInversion (A, low, mid)
    rightCount = CountInversion (A, mid+1, high)
    SplitCount = MergeAndCount (A, low, mid, high)
    return leftCount + rightCount + splitCount
FUNCTION: MergeAndCount (A, low, mid, high):
    Left = A [for count in range (low, mid)]
    Right = A [for count in range (mid+1, high)]
    i=0, j=0, count=0
    while i < length(Left) and j < length(Right):
        if Left[i] <= Right[j]:
            mergedArray.append (Left[i])
            i++ i = i+1
        else:
            mergedArray.append(Right[j])
            count = count + length(Left) - i
            j = j+1
```

```
while i < lenght(Left): and j < length(Right):
    A.append(Left[i])
    i++ i = i+1
while j < length(Right):
    A.append(Right[j]):
    j = j+1
return count
```

$$T(n) = 2T(n/2) + O(n)$$

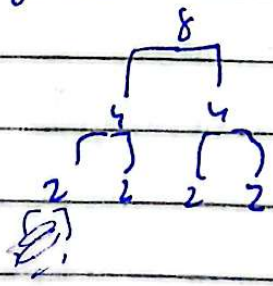
Applying master's theorem:-

$\log_2 2 = 1$, $k = 1$, $p = 0$

case 2 :- $p > -1$ hence $T(n) = \Theta(n \log n)$

(b) d) The best case time complexity of quick sort is $O(n \log n)$ which occurs when the pivot divides the array into two equal halves every time.

eg →



partitioning takes $O(n)$ and we have approximately $\log_2 n$ levels which means we have

$$T(n) = 2T(n/2) + O(n)$$

Date:

master' theorem:-

$k = \log_2 2 = 1$

case 2 :- $p = 0$, $p > -1$

$T(n) = n(\log n)$ $n \log n$

The worst case complexity is $O(n^2)$ which occurs when the pivot is the smallest or the largest element which leaves one subarray empty and the other having $n-1$ elements

eg A = 1 2 3 4 5

↓

2 3 4 5

↓

3 4 5

↓

4 5

↓

5

recurrence: $T(n) = T(n-1) + O(n)$

$T(n) = cn + c(n-1) + c(n-2) + \dots$

$T(n) = \frac{n(n+1)}{2} \rightarrow O(n^2)$

We have n levels and each level requiring linear partitioning work.

Date: 21/10/26.

Assignment 02 part (a):

1- Algorithm using divide & conquer:

Step#1:- If n=2, return the absolute difference of two numbers d= |A[0]-A[1]|

Step#2:- If n=1, return "no pair exist".

Step#3:- If n>2, then divide the array in approx 2 equal halves AL and AR.

Step#4:- Recursively find the minimum distance in the left half & store it in dL.

Step#5:- Recursively find the minimum distance in the right half & store it in dR.

Step#6:- find the smaller of 2 distances d= min (dL, dR).

Step#7:- Compare every number in the left half with every right half.

Step#8:- Return d, the minimum distance.

SP PAPER PRODUCT

CamScanner

Date: 2/10/24.

2) Pseudocode:

```
Closest-pair(A, n):
  if n == 2
    return A[0] - A[1]
  if n == 1
    return "No pair exist"
  if n > 2:
  Divide A into approx 2 equal halves.
    dL = Closest pair (AL, len(AL))
    dR = Closest pair (AR, len(AR)).
    d = min(dL, dR).
  for i = 0 to (AL)-1:
    for j = 0 to len(AR)-1.
      d = min (d, [A[i] - A[j]]).
  return d.
```

Recurrence: $T(n) = 2T\left(\frac{n}{2}\right) + cn^2$

$a = 2, b = 2,$

$f(n) = cn^2.$

$T(n) = O(n^2)$

↓

No it is not a good algorithm.

SP PAPER PRODUCT

CamScanner

Date: 21/10/26

:-) A more efficient approach is :-

Pseudocode :

```
Closest-pair(A, n):
    Sort(A)
    min_Diff = A[1] - A[0]
    for (i = 0 to n-2):
        d = A[i+1] - A[i]
        if d < min_Diff
            minDiff = d
    return min_Diff
```

2) Time-complexity :-

Sorting : O(n log n)

Comparing : O(n)

minimum-difference : O(n)

Overall => O(n log n) → which is less than $O(n^2)$ so this approach is better.

PAPER PRODUCT

CamScanner

Date: 21/10/22

2) part (f) : finding the peak in a Unimodal array.

→ Algorithm using divide & Conquer method:-

Pseudocode:-

```
find_peak(A, low, high):

    if low = high:
        return low

    mid = (high+low)/2
if A[mid] > A[mid-1] AND A[mid] > A[mid+1]
        return mid.

if A[mid] < A[mid+1]:
    return find_peak(A, mid+1, high)

else
    return find_peak(A, low, mid-1)
```

SP PAPER PRODUCT

CamScanner

Date: 21/10/22

1) Dry-Run:

Ex:- A = [3, 7, 11, 16, 14, 10, 5]
0 1 2 3 4 5 6

O16 = (3).

A[mid] = 16.

16 > 11 & 16 > 14 so peak-found.

p = 4 A[p] = 16 Ans

Time-Complexity:-

Recurrence-Relation: $T(n) = T(n/2) + O(1)$

Sol:-

$\frac{n}{2^k} = 1 \quad k = \log n$

$T(n) = O(\log n)$

Ans

PAPER PRODUCT

CamScanner

Date: 21/10/20

2) part (g) :- Max profit from buying & selling stock

2) Pseudocode :-

Using → Divide & Conquer

Case 1 / Step 1 :- Buy & sell entirely in the left half.
Case 2 / Step 2 :- Buy & sell entirely in the right half.
Case 3 / Step 3 :- Buy in the left half & sell in the right half.

=) Max_profit (P, low, high):

if low == high:
return (profit=0, buy=low, sell=high, minPrice = P[low], minDay=low, maxPrice = P[high], maxday = high)

mid = low + (low + start)/2 :
left = Max-profit (P, low, mid)
Right = Max-profit (P, mid+1, high).

→ find the minimum price & its day in P[low ... mid]
→ find the maximum price & its day in P[mid+1 ... high].

crossprofit = maxRight - minLeft.

best = Left.profit, right.profit, crossProfit

SP PAPER PRODUCT

CamScanner

Date: 21/10/24

Return best day with the min & max price & their corresponding days.

After the algorithm finishes:
if best profit <= 0:
Report "No profitable transaction exists"
else:
Report the buy day, sell day & max profit.

2) Dry Run:

Recurrence relation: $T(n) = 2T(n/2) + O(n)$

Soln:

$n^{\log_b a} = n^{\log_2 2} = n$

$T(n) = O(n \log n)$

SP PAPER PRODUCT

CamScanner

Date: 21/10/25

Dry Run :-

Ex- ps [10, 6, 9, 4, 12, 7]

[10, 6, 9] [4, 12, 7]

left-profit = 9-6 = 3.
Right-profit = 12-4 = 8
Cross-profit = 12-6 = 6

b) Buy on day 4 & sell on day 5
12-4 = 8
↳ max-profit.

c) The algorithm divides the days into two halves, recursively finds the best profit in each half & calculates the crossing profit using the minimum left-half price & max right-half price. It selects the best of these three options.

T.C = O(nlogn)

PART H (Done by Abdullah 24K0977)

Exercise 1 - Finding median of two Databases

↳ approach used to solve : Binary Search

↳ Pseudocode :

```
ALGORITHM findMedian(n) {
    low = 0
    high = n
    while low <= high
        i = floor((low + high)/2)
        j = n - i
    if i = 0 then
        leftA = -∞
    else
        leftA = Query(A, i)
    if i = n then
        rightA = +∞
    else
        rightA = Query(A, i+1)
    if j = 0 then
        leftB = -∞
    else
        leftB = Query(B, j+1)
```

```
if leftA <= rightB AND
   leftB <= rightA then
        return max(leftA, leftB)
else if leftA > rightB then
        high = i - 1
else
        low = i + 1
}
```

we have $i + j = n$ selected values. If both partition conditions hold, every selected value is less than every ~~no~~ nonselected value, and so the selected values are exactly the smallest n values accross both databases. The largest selected value is the nth smallest overall which is the required median.
If leftA > rightB, too many values have been selected from A so we reduce i, otherwise we increase i.

Complexities

① Database Queries : $O(\log n)$
② Extra space : $O(1)$

Exercise 2 - Count significant inversions

↳ can be tackled using a special merge sort algorithm. Divide the array into two halves and recursively count significant inversions in each half. Then count the significant inversions that cross between the halves. Since these halves are sorted by the recursive calls, a moving pointer can count these cross half pairs.

↳ALGORITHM CountSigInv(A, left, right) {

```
if left >= right
        return 0
mid = floor((left + right) / 2)
count = countSigInv(A, left, mid)
count = count + CountSigInv(A, mid+1, right)
j = mid+1
for i = left to mid
        while j <= right AND A[i] > 2*A[j]
                j = j+1
        count = count + (j - (mid+1))
Merge A[left..mid] & A[mid+1..right]
        in ascending order.
return count
```

}

↳ Consider example $A = [5, 1, 3, 2]$

The sig inversions would be:

PAIR	CHECK	SIG?
(1,2)	$5 > 2(1)$ or $5 > 2$	Yes
(1,3)	$5 > 2(3)$ or $5 > 6$	no
(1,4)	$5 > 2(2)$, or $5 > 4$	Yes
(2,3)	$1 > 2(3)$ or $1 > 6$	no
(2,4)	$1 > 2(2)$ or $1 > 4$	no
(3,4)	$3 > 2(2)$ or $3 > 4$	no

Total # of sig ~~inv~~ inversions = 2

↳ Complexity

Two recursive calls process halves of size $n/2$

Counting cross half pairs and merging will take $O(n)$ time in total so

$$T(n) = 2T(n/2) + O(n)$$

and by Master's Theorem:

$$T(n) = O(n \log n)$$

24k-0549 Date:

Q1 h) Ex 4:-

$F_j = C \cdot q_j (L_j - R_j)$

$L_j = \sum_{0 \le i < j} \frac{q_i}{(j-i)^2}$ → contribution of all particles from the left

$R_j = \sum_{i > j} \frac{q_i}{(j-i)^2}$ → contribution of all particles to the right

hence $F_j = C q_j (L_j - R_j)$.

Since both sums have the structure of convolution we will use Fast Fourier Transform (FFT) which allows us to multiple 2 polynomials in $O(n \log n)$ time.

Creating an array $k[]$ such that $k[0] = 0$ and $k[k] = 1/k^2$ for $k \ge 1$

Calculating convolution of charge array q and kernel $1/k$:

$L[j] = \sum_{i=0}^{j-1} \frac{q_i}{(j-i)^2}$

perform the same convolution on the right side by reversing the array right side array.

```
FUNC computeForces(q[]):
    create k[0..n-1]
    k[0] = 0
    for k in range(1, n-1) :-
        k[k] = 1/k*k
    L = FFT_convolution(q, k)
    qRev = Reverse(q)
    R = FFT_convolution(qRev, k)
    R = Reverse(R)
    for j = 1 to in range(1, n):
        ~~F[j]~~ F[j] = c * q[j] * (L[j] - R[j])
    return F

def FUNC FFT(A, inverse):-
    n = length(A)
    j = 0
    for i = 1 to n-1 :-
        bit = n/2
        while (j and bit) != 0:
            j = j xor bit
            bit = bit/2
        j = j xor bit
        if i < j:
            swap(A[i], A[j])
    length = 2
```

Date:

```
while length <= n.
    angle = 2 * PI / length
    if inverse == false:
        angle = -angle
    wlen = exp(i * angle)
    half = length / 2
    for start = 0 to n-1, step length:
        w = 1
        for k = 0 to half - 1:
            u = A[start + k]
            v = A[start + k + half] * w
            A[start + k] = u + v
            A[start + k + half] = u - v
            w = w * wlen
    length = length * 2
if inverse == true:
    for i = 0 to n-1:
        A[i] = A[i] / n
FUNC FFT_convolution(A, B):
    needed = length(A) + length(B) - 1
    size = 1
    while size < needed:
        size = size * 2
    ~~extend A~~ zero extend A until length(A) = size
    zero extend B until length(B) = size
```

Date:

FFT(A, false)
FFT(B, false)
for i=0 to size-1:
    A[i] = A[i] * B[i]
FFT(A, true)
Return first needed elements of A.

Time complexity:-
creating kernel for K : O(n)
first FFT convolution for L: O(n log n)
Reverse charges and calculate 2nd convolution: O(n log n)
Reverse the result & calculate final prices O(n)
Total: T(n) = O(n log n)